DERMS-R Full Benchmark Execution Plan

Docker (current) vs. Kubernetes vs. Ray (RTX 5000 Ada + RTX 5090) vs. Federated Learning — 10-epoch run, 20:00-08:00 operating window

DERMS-R / RIGAL-AI

2026-10-01

Contents

DERMS-R Full Benchmark Execution Plan **1**
1. Scope being estimated . . . 1
2. The single most important finding before trusting any time estimate . . . 2
3. Throughput model used . . . 3
3.1 Operating window: 20:00–08:00 only (12h/day) . . . 3
4. Approach A — Current approach (single-GPU Docker orchestrator) . . . 4
4.1 Setup & start training (Docker) . . . 4
5. Approach B — Multi-GPU orchestration across RTX 5000 Ada + RTX 5090 . . . 4
Kubernetes . . . 5
5.1 Cloud alternative: 8x A100 instead of physical RTX 5090s . . . 6
5.2 Cloud alternative: 16x RTX 4090 . . . 7
5.3 Orchestration alternative: Ray instead of Kubernetes . . . 7
6. Approach C — Federated learning . . . 9
7. Comparison summary (epochs=10) . . . 10
8. Recommendation . . . 11
9. Calibration step (do this before committing to a schedule) . . . 11
10. Key assumptions and confidence . . . 11

DERMS-R Full Benchmark Execution Plan

1. Scope being estimated

The canonical DERMS-R protocol (`README.md`, `derms/config.py`) defines:

Dimension	Value
Architectures	12 (`resnet50`, `efficientnet_b0`, `efficientnetv2_s`, `convnextv2_tiny`, `xception41`, `vit_base`, `deit3_base`, `swin_tiny`, `maxvit_tiny`, `nextvit_base`, `efficientvit_b0`, `mobilenetv4`)
Dataset ratios	5 (0.10, 0.25, 0.50, 0.75, 1.00)
CV folds	5 (StratifiedKFold, seed 42)
Datasets	5 (`cifar10`, `cifar100`, `flowers102`, `food101`, `all` / Blood Cell Cancer ALL)
Epochs per fit	**10** (reduced from the README's canonical 50 — see caveat below)
Fits (dev candidate search)	$12 \times 5 \times 5 \times 5 =$ **1,500**

Dimension	Value
Locked final refits	+5 (one per dataset winner)
Total fits (training runs)	**1,505**
Total orchestration units (K8s Jobs / Ray tasks)	**65** (60 train + 5 lock — see note below)
Training precision	Full-fine-tune FP32, **TF32 disabled**, deterministic kernels (`derms/utils.py`: `allow_tf32=False`, `use_deterministic_algorithms(True)`)

> **Fits vs. orchestration units:** `train.py` has no `--ratios` flag — every invocation already runs all 5 canonical ratios × 5 folds (25 fits) internally for whichever `--models`/`--datasets` it's given. So while the benchmark is still 1,505 individual training fits, the natural container/Job/task unit is one per (model, dataset) pair: 12 models × 5 datasets = 60 train units, plus one lock/refit unit per dataset once that dataset's 12 models finish = **65 units total**. This doesn't change the total compute or busy-GPU-hours below (same 1,505 fits either way), but it does mean the Kubernetes and Ray sections (§5) orchestrate 65 Jobs/tasks, not 1,505 — a deployable implementation of this is in `k8s/` in this repository (`k8s/README.md`).
>
> **Caveat on epochs=10:** the README's canonical protocol specifies 50 epochs with cosine decay and min-validation-loss checkpoint selection. Cutting to 10 epochs is a *training-budget* change, not one of the protected DERMS-R constants (`alpha/beta/gamma/delta/P0/sigma0`/ratios), so it doesn't invalidate the metric definition — but it does risk under-training the heavier/slower-converging architectures (ViT-Base, DeiT3-Base, Swin, ConvNeXtV2), which could distort the accuracy term `\bar{A}_{i,r}` and therefore architecture rankings. Treat a 10-epoch run as a fast sensitivity/dry-run pass to validate the pipeline and get directional rankings, not as a substitute for the canonical 50-epoch numbers reported as final results.

Development-set sizes used for the estimate (`N_dev`, after held-out test splits):

Dataset	N_dev
cifar10	50,000
cifar100	50,000
flowers102	2,040
food101	75,750
all (Blood Cell Cancer)	2,918 (3,242 raw × 90%, per `prepare_dataset.py --test_ratio 0.10`)

Total work = ~**225.5 million** image forward+backward passes across the full matrix (12 models × 5 ratios × 5 folds × 10 epochs × 5 datasets, train-split = 80% of each ratio-subset per fold) — exactly 1/5 of the 50-epoch total (~1.13 billion), since epoch count is a linear multiplier on compute.

2. The single most important finding before trusting any time estimate

`derms/utils.py` explicitly sets:

```
torch.backends.cuda.matmul.allow_tf32 = False
torch.backends.cudnn.allow_tf32 = False
torch.use_deterministic_algorithms(True)
```

This is **true FP32** training with deterministic kernels — not the TF32-accelerated "FP32" mode most public PyTorch throughput numbers quietly assume. On Ada/Blackwell tensor-core GPUs this is commonly **2.5–3x slower** than default settings, because matmuls/convs can't use the TF32 tensor-core path and some ops fall back to slower deterministic implementations. This is a deliberate, correct choice for reproducible CV stability (`s_{i,r}`) — it should **not** be changed to hit a deadline, because it would silently invalidate

the stability term the whole benchmark depends on. But it means naive GPU-spec-sheet estimates will be badly wrong if this factor is ignored.

All estimates below bake in a **2.8x derate** versus default-TF32 throughput for this reason. This is still a modeled number, not a measurement — see the calibration step in §9.

3. Throughput model used

Reference images/sec at batch=32, 224px, full fine-tune, default (TF32-on) PyTorch settings on an RTX-4090-class card (synthesized from known relative model cost; not measured in this repo), then scaled:

- **÷2.8** for TF32-off + deterministic kernels (this repo's actual config)
- **×0.70** for RTX 5000 Ada (vs. the RTX-4090 reference; blend of FP32-TFLOPS ratio 65/82.6 and bandwidth ratio 576/1008 GB/s)
- **×1.50** for RTX 5090 (vs. the same reference; blend of FP32-TFLOPS ratio 104.8/82.6 and bandwidth ratio 1792/1008 GB/s)
- **×1.13** for A100 (blend of FP32 CUDA-core ratio 19.5/82.6 and HBM2e bandwidth ratio 2039/1008)
- **×1.00** for RTX 4090 (the reference card itself)

Resulting modeled throughput (img/s, this repo's actual training config — unaffected by the epoch count or the operating window below):

GPU	Avg across 12 models	Lightest model	Heaviest model
RTX 5000 Ada	~103 img/s	~225 (mobilenetv4)	~55 (vit_base/deit3_base)
RTX 4090	~147 img/s	~321 (mobilenetv4)	~79 (vit_base/deit3_base)
A100	~166 img/s	~363 (mobilenetv4)	~89 (vit_base/deit3_base)
RTX 5090	~221 img/s	~482 (mobilenetv4)	~118 (vit_base/deit3_base)

A **1.3x wall-clock overhead** is applied on top of raw compute time for data loading, validation passes, checkpoint I/O, and MLflow logging across the 1,505 fits, plus container start/stop between the much smaller number of actual container launches (65 — see the fits-vs-orchestration-units note in §1, since `train.py` loops over all 25 ratio×fold fits per container invocation). This gives the **busy GPU-hours** figure used throughout — the total time the GPU(s) must actually be running.

3.1 Operating window: 20:00–08:00 only (12h/day)

The physical on-prem GPUs (RTX 5000 Ada, and any RTX 5090 added to that pool) are assumed to run only during a nightly **20:00–08:00 window (12 of 24 hours/day)**, not continuously. This halves effective daily throughput versus a 24/7 run, so:

```
calendar days = busy GPU-hours / 12   (physical on-prem pool)
calendar days = busy GPU-hours / 24   (cloud-rented pool, assumed continuous)
```

Assumption flagged: cloud-rented GPUs (8x A100, 16x RTX 4090 — §5.1/§5.2) are modeled as running continuously, since they're billed by the hour with no reason to idle them inside a window tied to a specific physical location. If the 20:00–08:00 window should also apply to cloud usage, the calendar duration for those options doubles too (the dollar cost is unchanged either way — the same number of GPU-hours gets billed, just spread over more calendar days).

Operational requirement: jobs that don't finish inside one nightly window must pause and resume cleanly at the 08:00/20:00 boundary. `train.py --resume` (already documented in `README.md`) handles this at the

process level; the orchestrator (or Ray driver, §5.3) needs a scheduler that sends a clean stop signal at 08:00 and restarts the queue at 20:00 for whichever job was mid-flight.

4. Approach A — Current approach (single-GPU Docker orchestrator)

What exists today: the orchestrator (`orchestrator/`) launches one `derms-r-train:cuda` container per job on a single host (currently the RTX 5000 Ada box, `edramos@10.175.190.224`), sequentially or a few at a time via Range mode (max 12 concurrent containers, but they'd all contend for the **same single GPU**, so real throughput stays ~flat, not 12x).

Estimated calendar time under the 20:00–08:00 window (epochs=10): ~65.9 days (791 busy GPU-hours ÷ 12h/day). For reference: ~33.0 days if this GPU instead ran continuously (24/7), and ~329.7 days under the 12h/day window at the canonical 50 epochs.

Pros: - Zero new infrastructure; works today. - Simplest to debug and audit (one host, one log stream, MLflow already wired up).

Cons: - ~66 days under the nightly window is over two months for a 10-epoch dry run; any host downtime, driver update, or crashed container directly extends the calendar schedule, and there's no parallelism across models/datasets since everything is strictly serial on the one GPU.

4.1 Setup & start training (Docker)

On the RTX 5000 Ada box (`edramos@10.175.190.224`), from the repo root:

```
# One-time: build the CUDA training image and bring up the orchestrator + MLflow
docker-compose -f docker-compose.yml -f docker-compose.gpu.yml --profile train build train
docker-compose -f docker-compose.yml -f docker-compose.gpu.yml up -d orchestrator mlflow
```

Open `http://<host>:8080` (or `ssh -L 8080:localhost:8080 edramos@10.175.190.224` if accessing remotely), log in with the one-time password from `docker logs <orchestrator-container>`, and start training from the Configuration page — or skip the UI and launch a run directly:

```
docker run -d --name derms-train-cifar10-resnet50 \
  --gpus all --shm-size 2g --network derms-r \
  -v "$(pwd)/datasets:/workspace/datasets" \
  -v "$(pwd)/results:/workspace/results" \
  -v "$(pwd)/hf-cache:/root/.cache/huggingface" \
  -e HF_HOME=/root/.cache/huggingface \
  derms-r-train:cuda train.py --data /workspace/datasets --output /workspace/results \
  --datasets cifar10 --models resnet50 --epochs 10 --batch-size 32 --device cuda \
  --resume --download --mlflow --mlflow-tracking-uri http://mlflow:5000 --mlflow-experiment derms-r-ben
```

Repeat per (model, dataset) pair (60 combinations for the dev search, then `--lock <dataset>` once each dataset's 12 models finish — see `README.md`'s "Run, resume, lock, evaluate"). This is exactly the single-container, single-GPU, fully-serial path §4 estimates — there's no parallelism to configure here, which is both its simplicity and its ~66-day cost.

5. Approach B — Multi-GPU orchestration across RTX 5000 Ada + RTX 5090

Two orchestration options are compared for running the job matrix across a pool combining the existing RTX 5000 Ada with an RTX 5090: **Kubernetes** (§5 body) and **Ray** (§5.3), plus two cloud-rental alternatives that sidestep the physical pool entirely (§5.1, §5.2).

> **Deployable packages:** working implementations of both orchestration options live in this repository — `k8s/` for Kubernetes (both training images, PVCs, MLflow, the 65-Job generator with load-balanced GPU assignment, the 20:00-08:00 suspend/resume CronJobs, and Grafana/DCGM GPU+system monitoring; see `k8s/README.md`) and `ray/` for Ray (head/worker start scripts +

a driver implementing the same 65-task matrix and operating window without Kubernetes; see `ray/README.md`). "Setup & start training" subsections below give the condensed command sequence for each.

Kubernetes

Proposed architecture: - A small K8s cluster (even 1-2 nodes is enough) with the **NVIDIA device plugin** and **node labels** per GPU model (`nvidia.com/gpu.product=RTX-5000-Ada` / `RTX-5090`), so workload scheduling can target the right image per node. - **Two training images**, not one: the existing `Dockerfile` (`torch==2.4.1+cu121`) targets Ada (sm_89) and already works on the RTX 5000 Ada box. **RTX 5090 is Blackwell (sm_120) and this image will not run on it** — it needs a second image built on a CUDA 12.8+/PyTorch 2.7+ base with Blackwell kernel support. This is a real blocker to resolve before any RTX 5090 node joins the pool, not just a config tweak. - **One K8s `Job` per (model, dataset) pair, not per fit** — `train.py` has no `--ratios` flag, so one invocation already runs all 5 ratios × 5 folds (25 fits) internally. That's 12 models × 5 datasets = **60 train Jobs**, each with `resources.limits: {nvidia.com/gpu: 1}` and node affinity to the correct GPU pool, plus **5 lock/refit Jobs** (one per dataset) that wait for their dataset's 12 train Jobs to reach `Complete` before running `train.py --lock`. - **No separate workflow engine (Argo/Kubeflow) needed at this scale.** 65 Jobs with a shallow one-level dependency (5 lock Jobs each waiting on 12 train Jobs) is comfortably handled by plain Kubernetes Jobs plus a small RBAC'd `initContainer` that polls the Jobs API for its dataset's train Jobs to complete — that's what `k8s/60-jobs/` actually implements. The 20:00–08:00 window uses Jobs' native `spec.suspend` field (stable since K8s 1.21) via two CronJobs, not a workflow-engine-specific pause mechanism. - Centralize artifacts: MLflow server already exists; move result/checkpoint storage from host bind-mounts to a PVC or S3-compatible object store (MinIO) so any node can write results.

Estimated time at epochs=10, scaling the GPU pool, under the 20:00–08:00 window:

Pool	Busy GPU-hours	Continuous (24/7)	**Calendar @ 12h/day**
1x RTX 5000 Ada (= Approach A)	791 h	33.0 d	65.9 d
2 GPU: 1x RTX 5000 Ada + 1x RTX 5090	274 h	11.4 d	**22.8 d (~3.3 weeks)**
4 GPU: 2x RTX 5000 Ada + 2x RTX 5090	143 h	6.0 d	**11.9 d (~1.7 weeks)**
8 GPU: 2x RTX 5000 Ada + 6x RTX 5090	65 h	2.7 d	**5.4 d**

(Parallel efficiency assumed 92%/88%/82% at 2/4/8 GPUs, for scheduling overhead and shared resources. At the canonical 50 epochs, multiply every column by 5.)

Setup & start training (Kubernetes) Full deployable package in `k8s/` — condensed version of `k8s/README.md`:

```
# 0. No Kubernetes cluster yet? Stand one up with k3s (k8s/00-cluster-install/):
#    on the RTX 5000 Ada box:
./k8s/00-cluster-install/install-k3s-server.sh    # prints K3S_URL + K3S_TOKEN
#    on the RTX 5090 box:
K3S_URL=https://<server-ip>:6443 K3S_TOKEN=<token> ./k8s/00-cluster-install/install-k3s-agent.sh

# 1. GPU nodes: NVIDIA Container Toolkit registered with k3s's bundled
#    containerd (k3s uses its own path, not the system default), device
```

```
#    plugin, node labels (k8s/30-gpu-setup/README.md)
sudo nvidia-ctk runtime configure --runtime=containerd \
  --config=/var/lib/rancher/k3s/agent/etc/containerd/config.toml.tmpl --set-as-default
helm install nvdp nvdp/nvidia-device-plugin --namespace nvidia-device-plugin --create-namespace
kubectl label node <rtx-5000-ada-node> gpu-arch=ada
kubectl label node <rtx-5090-node>     gpu-arch=blackwell

# 2. Static scaffolding: namespace, PVCs, MLflow, RBAC, the 20:00/08:00 scheduler
#    (edit the REPLACE_ME_* placeholders in 10-storage/*.yaml and
#    70-scheduling/operating-window-cronjobs.yaml first)
kubectl apply -k k8s/

# 3. Seed the datasets/hf-cache PVCs (k8s/README.md's "Seeding data" section),
#    then build + push both training images
REGISTRY=ghcr.io/you k8s/40-images/build-and-push.sh

# 4. Generate and apply the 65-Job matrix (epochs=10 dry run first)
python3 k8s/60-jobs/generate_manifests.py --registry ghcr.io/you --epochs 10
kubectl apply -k k8s/60-jobs/generated/

# 5. Watch it run
kubectl get jobs -n derms-r -L derms.gpu-arch -w
```

k3s was chosen over kubeadm/vanilla k8s specifically for this 2-node, single-purpose cluster: single-binary install, no separate etcd to operate, and both nodes still run training Jobs (k3s doesn't taint the server node). The one real consequence is the k3s-specific containerd path in step 1 above — everything else (device plugin, Helm charts, the Job manifests) is unmodified, conformant Kubernetes either way.

Re-run step 4 with `--epochs 50` into a fresh output path once the dry run validates end-to-end. GPU/system monitoring (`k8s/80-monitoring/`) and the optional ALL-dataset prep job (`k8s/50-prepare/`) are also covered in `k8s/README.md`.

5.1 Cloud alternative: 8x A100 instead of physical RTX 5090s

A100 (sm_80) is fully supported by the **existing** `torch==2.4.1+cu121` image — no Blackwell rebuild needed. But its big architectural advantage (TF32/FP16/BF16 tensor-core throughput) is exactly what `derms/utils.py` disables for determinism, so you only get its raw CUDA-core FP32 rate. Per GPU this lands **between RTX 5000 Ada and RTX 5090** (blended scale 1.13x the RTX 4090 reference, vs. 0.70x for RTX 5000 Ada and 1.50x for RTX 5090). Modeled as continuous/24-7 (§3.1):

Pool	Busy GPU-hours	Continuous (24/7)
1x A100	377 h	15.7 d
2x A100	199 h	8.3 d
4x A100	103 h	4.3 d
8x A100	68 h	**2.8 d**

Estimated cloud cost for the full 1,505-fit benchmark at epochs=10 (~68 busy GPU-hours on 8 GPUs): **~$820 at cheap/spot pricing (~$1.50/GPU-hr) up to ~$2,230 at on-demand pricing (~$4.10/GPU-hr)** — 1/5 of the 50-epoch cost, since GPU-hours scale linearly with epochs. If the 20:00–08:00 window were applied here too, calendar time would double to ~5.7 days at the same dollar cost. Treat the cost range as approximate and re-check current provider pricing before committing.

Setup & start training (8x A100) Not a separate technology stack — rent 8 A100s from any GPU cloud (Lambda, CoreWeave, RunPod, AWS p4d, etc.), then run **either** of the two paths already built, unmodified,

since A100 is sm_80 and needs no new image:

```
# Path 1: join the rented A100s as Kubernetes nodes to the existing k8s/ package
#   (install the NVIDIA Container Toolkit + device plugin on each, label
#   gpu-arch=a100, build k8s/60-jobs/generate_manifests.py's GPU_POOL to add
#   an "a100" entry at relative_speed=1.13 alongside ada/blackwell)
kubectl label node <a100-node-1> gpu-arch=a100
# ... then the same kubectl apply -k k8s/ flow as the Kubernetes setup above.

# Path 2 (simpler for a short-lived rental): plain docker run on each instance,
# same as Approach A's single-GPU command, pointed at the existing Ada image
# (torch 2.4.1+cu121 already supports sm_80) -- one (model, dataset) pair per
# instance, run in parallel across all 8 rented GPUs:
docker run -d --gpus all --shm-size 2g \
  -v "$(pwd)/datasets:/workspace/datasets" -v "$(pwd)/results:/workspace/results" \
  derms-r-train:ada train.py --data /workspace/datasets --output /workspace/results \
  --datasets cifar10 --models resnet50 --epochs 10 --batch-size 32 --device cuda --resume --download
```

Path 2 needs the same shared `results//datasets/` storage discipline as Ray (§5.3) since there's no PVC — mount a shared NFS export or sync `results/` back centrally before locking each dataset.

5.2 Cloud alternative: 16x RTX 4090

RTX 4090 is the architecture this whole estimate is scaled from (reference = 1.0x), so with the repo's TF32-off/deterministic config it runs at **147.0 img/s/GPU** — between RTX 5000 Ada (102.9) and A100 (166.1), below RTX 5090 (220.5). It is Ada Lovelace (sm_89), the same family as RTX 5000 Ada, so it also needs **no image changes**. Modeled as continuous/24-7 (§3.1):

Pool	Busy GPU-hours	Continuous (24/7)
1x RTX 4090	426 h	17.8 d
2x RTX 4090	224 h	9.3 d
4x RTX 4090	116 h	4.8 d
8x RTX 4090	59 h	2.5 d
16x RTX 4090	41 h	**1.7 d**

Still the fastest and cheapest option modeled: 16x RTX 4090 beats even 8x A100 on wall-clock (1.7 d vs. 2.8 d). Estimated cloud cost for the full run at epochs=10 (~41 busy GPU-hours on 16 GPUs, at consumer-card cloud rates of ~$0.35-$0.60/GPU-hr): ~**$230-$390**. If the 20:00–08:00 window were applied here too, calendar time would double to ~3.4 days at the same dollar cost.

Setup & start training (16x RTX 4090) Same as the 8x A100 setup above — rent from a consumer-card-friendly provider (vast.ai, RunPod, Lambda), then either join the instances as Kubernetes nodes (RTX 4090 is Ada Lovelace/sm_89, so it uses the **same image** as the RTX 5000 Ada pool — no third image needed, just label the nodes `gpu-arch=ada` and let `k8s/60-jobs/generate_manifests.py` schedule onto them) or run plain `docker run` commands per (model, dataset) pair across the 16 rented GPUs as in the A100 Path 2 example.

5.3 Orchestration alternative: Ray instead of Kubernetes

For just 2-8 physical GPUs, a full Kubernetes control plane + NVIDIA device plugin is more cluster machinery than the job actually needs — even without a workflow engine (§5 already notes 65 Jobs doesn't need Argo/Kubeflow either). **Ray** (`ray up` on bare-metal/SSH nodes, or `KubeRay` if a K8s control plane already exists) reaches the same outcome with less to stand up: a head node plus GPU worker nodes, **placement**

groups to pin each job to a specific physical GPU, Ray's built-in task retry/fault-tolerance instead of hand-rolled `backoffLimit` logic, and a driver script that submits one `ray.remote` task per (model, dataset) pair — the same 60 train tasks + 5 lock tasks as the Kubernetes Job count in §5, each wrapping the existing `train.py` CLI and internally running its 25 ratio×fold fits. A simple cron-style check in the driver (stop submitting new tasks and checkpoint in-flight ones at 08:00, resume the queue at 20:00) covers the operating window without any Kubernetes `CronJob`/RBAC machinery.

Ray does **not** remove the RTX 5090 Blackwell image blocker — that is a CUDA/PyTorch build issue independent of the orchestrator, so the same two-image strategy from §5 (Ada image for RTX 5000 Ada, a CUDA 12.8+/PyTorch 2.7+ image for RTX 5090) is still required.

Raw per-GPU throughput is identical to the Kubernetes numbers above (same hardware, same `derms/utils.py` config); the difference is operational overhead. Modeled here with a modest efficiency gain from the simpler stack (95%/90%/85% vs. Kubernetes' 92%/88%/82% at 2/4/8 GPUs):

Pool	Busy GPU-hours	Continuous (24/7)	**Calendar @ 12h/day**
2 GPU: 1x RTX 5000 Ada + 1x RTX 5090	265 h	11.0 d	**22.1 d**
4 GPU: 2x RTX 5000 Ada + 2x RTX 5090	140 h	5.8 d	**11.7 d**
8 GPU: 2x RTX 5000 Ada + 6x RTX 5090	63 h	2.6 d	**5.2 d**

Ray is the lower-effort choice if the goal is just "run these 65 Jobs (1,505 fits) reliably across 2-8 GPUs" without needing a Kubernetes control plane for anything else. Kubernetes earns its extra complexity back if a K8s platform already exists for other workloads, or if multi-tenant/cluster-wide scheduling is needed beyond this one benchmark.

Setup & start training (Ray) Deployable package in `ray/` (`ray/README.md` for full detail):

```
# 1. Head node (no GPU needed)
RAY_HEAD_IP=<note this node's IP> ./ray/start_head.sh

# 2. A worker on each GPU box, inside the matching training image
#    (RTX 5000 Ada):
RAY_HEAD_IP=<head-ip> GPU_ARCH=ada \
  DATASETS_DIR=/path/to/shared/datasets RESULTS_DIR=/path/to/shared/results \
  ./ray/start_worker.sh
#    (RTX 5090):
RAY_HEAD_IP=<head-ip> GPU_ARCH=blackwell \
  DATASETS_DIR=/path/to/shared/datasets RESULTS_DIR=/path/to/shared/results \
  ./ray/start_worker.sh

# 3. Run the driver (epochs=10 dry run first)
python3 ray/driver.py --epochs 10 --mlflow-uri http://<mlflow-host>:5001
```

`driver.py` submits the same 60 train + 5 lock tasks as the Kubernetes generator, load-balanced the same way, and implements the 20:00-08:00 window itself (terminates and retries a task if the window closes mid-run, rather than relying on a Kubernetes-specific `spec.suspend`) — pass `--no-window` to run continuously instead. Needs shared storage reachable at the same path on both GPU boxes (NFS or kept-in-sync directories), since there's no PVC equivalent outside Kubernetes.

Pros (both Kubernetes and Ray): - Roughly linear speedup with GPU count; straightforward to burst onto cloud GPU nodes (A100/H100/L40S) for the same job definitions if more throughput is needed later. - Retries, partial-failure recovery, and auditability (per-job logs/status) come largely for free instead of being hand-rolled in the orchestrator.

Cons: - Real upfront engineering cost either way: multi-node setup, GPU scheduling, a second CUDA/PyTorch image for Blackwell, and migrating result storage off host bind-mounts. Ray's footprint is

smaller, but neither is a config flag on the current repo. - Needs the RTX 5090 hardware actually racked and joined to the pool; until then both degrade to Approach A's single-GPU number.

6. Approach C — Federated learning

Federated learning (e.g., FedAvg across simulated clients) is a training-algorithm change, not primarily an infrastructure change, and it does not map cleanly onto the canonical DERMS-R protocol:

- DERMS-R's stability term `s_{i,r}` is defined as the sample SD of validation accuracy across 5 **centralized** CV folds trained under one fixed, identical protocol. Introducing FL means each "fold" would need to become a federation of simulated clients with its own partitioning, communication rounds, and aggregation — a protocol change to the benchmark itself, not just an execution detail. This needs an explicit design decision (and likely a README amendment) before any FL run could be called DERMS-R-compliant, since the current README says not to silently alter the measurement protocol.
- Federated training typically needs **more total passes over the data to reach the same accuracy** as centralized training (client drift / partial participation / non-IID effects), commonly 1.5–2.5x the effective epoch-equivalents. At only 10 epochs of local budget to begin with, this convergence penalty is a bigger relative risk than at 50 epochs.
- Running FL "for real" across distributed clients would also add network communication rounds; even simulating it on one GPU cluster (the realistic option here) adds synchronization/aggregation overhead per round, estimated at 1.1–1.3x.

Estimated time at epochs=10, run on the physical Approach B 2-GPU pool under the 20:00–08:00 window:

Scenario	Multiplier vs. Approach B (2 GPU)	Busy GPU-hours	Continuous (24/7)	**Calendar @ 12h/day**
Optimistic (1.5x convergence, 1.1x comms)	1.65x	452 h	18.8 d	37.6 d
Central estimate (2.0x convergence, 1.2x comms)	2.4x	657 h	27.4 d	**54.7 d**
Pessimistic (2.5x convergence, 1.3x comms)	3.25x	889 h	37.1 d	74.1 d

Add **2-4 weeks of protocol design/engineering time** on top (defining client partitioning per ratio, aggregation rule, and how `s_{i,r}` is computed under FL) before this clock even starts — this fixed cost does not shrink with fewer epochs or a shorter window, so it increasingly dominates the schedule.

Setup & start training (prototype only — not a ready package) Unlike Docker/Kubernetes/Ray above, there is **no deployable package for this in the repository**, and §8's recommendation is to skip it unless there's a specific federated/edge requirement — building one out fully would contradict that recommendation. If a prototype is still warranted to validate the protocol-redesign questions above before committing real time to it, the conceptual shape would be:

1. Pick a federated framework that can wrap existing PyTorch training code — Flower is the common choice for simulating FedAvg across N clients on a single machine or small cluster without needing real edge devices.

2. Define the client partitioning scheme for one (model, ratio) cell: e.g., split that ratio's training fold into N IID or non-IID shards, one per simulated client — this is the protocol decision flagged above, not a configuration default.
3. Replace the centralized training step inside one CV fold with `N` local `Flower` client updates + one `FedAvg` aggregation round, repeated for enough rounds to reach convergence; keep the existing 5-fold CV structure and `derms.metric.derms_r` scoring around that substitution so `s_{i,r}` is still computed the canonical way across folds.
4. Only after that substitution is validated on one architecture/dataset/ratio cell would it make sense to generalize it across the full 12×5×5×5 matrix — at which point the Kubernetes or Ray packages above could host the simulated clients as GPU workers, reusing their image/scheduling pieces.

This is deliberately left as a design sketch, not code, consistent with §8's recommendation not to invest here without a concrete need.

Pros: - Only relevant if there's an actual requirement to benchmark architectures under a federated/edge-deployment constraint (e.g., DERMS use case with distributed grid-edge data that can't be centralized).

Cons: - Slower than Approach B on the same hardware and window, and requires redefining part of the canonical metric before results would be comparable to the existing DERMS-R numbers. Not recommended as a way to produce the standard benchmark faster or more cheaply.

7. Comparison summary (epochs=10)

Approach	Infra effort to stand up	Estimated time (65 Jobs / 1,505 fits, 10 epochs)	Canonical DERMS-R as-specified?
A. Current Docker, 1x RTX 5000 Ada	None (exists today)	~65.9 days (20:00-08:00 window)	Yes*
B. Kubernetes, 2 GPU (RTX 5000 Ada + RTX 5090)	Medium (cluster + 2nd CUDA image; 65 Jobs, no DAG engine needed)	~22.8 days (window)	Yes*
B. Kubernetes, 4-8 GPU (RTX mix)	Medium + more hardware/cloud burst	~5.4-11.9 days (window)	Yes*
B. Ray, 2 GPU (RTX 5000 Ada + RTX 5090)	Low-medium (lighter than K8s)	~22.1 days (window)	Yes*
B. Ray, 4-8 GPU (RTX mix)	Low-medium	~5.2-11.7 days (window)	Yes*
B. Cloud, 8x A100	Low-medium (rentable, no Blackwell image needed; ~$820-$2,230)	~2.8 days (24/7)	Yes*
B. Cloud, 16x RTX 4090	Low (rentable, same Ada image family as RTX 5000 Ada; ~$230-$390)	**~1.7 days (24/7)**	Yes*
C. Federated learning	High (new training algorithm + protocol redesign)	~37.6-74.1 days (window)	Only after protocol changes — not directly comparable

* "Yes" means the job matrix and DERMS-R equation are unchanged; it does **not** mean 10-epoch accuracy numbers are interchangeable with the README's canonical 50-epoch results (see §1 caveat). "Window" = 20:00-08:00, 12h/day (§3.1); cloud options are modeled as continuous/24-7.

8. Recommendation

1. **Run the calibration step below first** — it's a few minutes of GPU time and converts every number above from "modeled" to "measured, rescaled."
2. The 20:00–08:00 window roughly doubles every physical on-prem schedule versus running continuously, which makes **cloud rental comparatively more attractive if speed matters**: 16x RTX 4090 (~1.7 days, ~$230-$390) or 8x A100 (~2.8 days, ~$820-$2,230), both unaffected by the window since they're rented by the hour.
3. If keeping the workload on owned RTX 5000 Ada + RTX 5090 hardware under the nightly window is a hard requirement, **Ray is the lower-effort way to get there** for 2-8 GPUs (~22.1 days at 2 GPU, ~5.2-11.7 days at 4-8 GPU) versus standing up full Kubernetes + Argo for the same throughput. Kubernetes is still the better choice if a K8s platform already exists for other workloads.
4. Use epochs=10 as a pipeline/sensitivity dry run only; commit to the canonical 50-epoch run (multiply every figure above by 5) for any numbers that will be reported as final DERMS-R results.
5. Skip Approach C unless there is a specific product requirement for federated/edge training; under the nightly window it is now ~38-74 days on top of the protocol-redesign cost, and still isn't directly comparable to the canonical DERMS-R numbers.

9. Calibration step (do this before committing to a schedule)

On the real RTX 5000 Ada box. Note `train.py` has **no `--ratios` flag** — every invocation always runs all 5 canonical ratios × 5 folds internally (25 fits), so this command is itself a small 25-fit batch at `--epochs 2`, not a single fit:

```
python train.py --data ./datasets --output ./results/_calib \
  --datasets cifar10 --models mobilenetv4 \
  --epochs 2 --batch-size 32 --device cuda
```

Read the per-epoch duration that `train.py` already logs to MLflow for the **r=0.10, fold 0** run specifically (the smallest, fastest of the 25), compute measured img/s = `50000*0.10*0.8 / epoch_seconds`, and multiply every estimate in this document by `measured_img_s / 102.9` (the RTX 5000 Ada modeled average from §3). Repeat once on the RTX 5090 image when it exists, scaling by `/ 220.5`. This calibration run's own `--epochs 2` is independent of both the benchmark's epoch count (10 vs. 50) and the 20:00–08:00 window — it only measures per-epoch throughput.

10. Key assumptions and confidence

- Per-model throughput figures are engineering estimates derived from publicly known relative model cost (GFLOPs/architecture family), not measurements of this codebase on this hardware. Treat all time figures as **±40-50% uncertain** until §9's calibration is run.
- Reducing epochs from 50 to 10 scales every busy-GPU-hours and cost figure by exactly 1/5, since epoch count is a linear multiplier on total compute in this model; it does not change relative GPU rankings or the Blackwell/TF32 findings above.
- The 20:00–08:00 operating window is assumed to apply only to the physical on-prem GPUs (RTX 5000 Ada, RTX 5090), not to rented cloud GPUs (§3.1) — confirm this split is actually what's intended, since it materially changes which option looks fastest.
- RTX 5090 is assumed rack-able and network-reachable; it is not currently part of the verified GPU host (memory: only the RTX 5000 Ada box at `10.175.190.224` has been validated end-to-end).
- Dataset dev-set sizes for `flowers102` and `food101` use known torchvision split sizes (datasets not yet downloaded in this repo checkout); `cifar10`/`cifar100` sizes were confirmed from the local `datasets/` directory; `all` was computed from the local raw image count (3,242) and the documented 10% test holdout.
- Wall-clock overhead (1.3x) and parallel efficiency (92/88/82% for Kubernetes, 95/90/85% for Ray) are standard planning assumptions for container-orchestrated ML batch jobs, not measured on this cluster.